DEVELOPER PROJECT REPORT

QT PDF Editor

A browser-based PDF editor, and an honest look at how it's built

Repository: quantiradev/qt_pdf-editor • Branch: sprint1
Report Date: July 13, 2026
Classification: Internal – Engineering Documentation

Document Control

Revision History

Version	Date	Author	Description
0.1	Sprint 1	Project Team	Initial internal draft – auth, upload, basic viewer
1.0	2026-07-13	Engineering	Full rewrite: architecture, API reference, security, testing, deployment, roadmap

A Quick Note on Sources

This report is based on a direct read of the sprint1 branch – the actual frontend and backend source, the dependency manifests, and the commit history, not just a summary of the project. There was an earlier draft of this report sitting in the repo, but only its Word lock file survived (the placeholder file Word creates while a document is open), with no recoverable text inside it. So rather than merge two drafts together, this one was rebuilt from scratch, straight from the code. If the earlier draft turns up separately, it's worth folding into a future revision.

1. Executive Summary

QT PDF Editor is a browser-based PDF editor: upload a file, and you can mark it up, restructure it, and save real changes back into it without ever leaving the tab. Text edits, highlights, ink, shapes, images, links, sticky notes, form fields, watermarks, page reordering, rotation, splitting, merging – it's all there, backed by proper undo/redo rather than a single "are you sure" dialog.

Under the hood it's a fairly conventional two-tier setup: Next.js and React on the front end, rendering pages through PDF.js and keeping editor state in a Zustand store, talking to a FastAPI backend that does the actual work of mutating the PDF via PyMuPDF. The one genuinely distinctive piece is a hand-built paragraph reflow engine that lets you edit text in place and have it re-wrap using the document's own fonts – most PDF editors fake this with an overlay box; this one actually rewrites the page content.

For a project at sprint 1, the feature set is impressively deep. It's worth being equally honest about where things stand operationally, though: everything currently runs as a single local process, metadata lives in a flat JSON file, there's no automated testing yet, and there's no CI/CD. None of that is unusual this early on, but it is the gap between where the code is today and where it would need to be to run in production – and Sections 7 through 10 walk through what closing that gap looks like.

What's Working Well

- Real client-side rendering (PDF.js) paired with real server-side, format-accurate editing (PyMuPDF) – edits land in actual PDF structures, not a rasterized layer stuck on top.
- The paragraph reflow engine (text_engine.py) redacts and re-wraps text using the document's own embedded fonts, so an edited paragraph still looks like it belongs in the document.
- Per-file locking plus version-stamped snapshots give you multi-step undo/redo at the whole-document level, not just "undo the last annotation."

- The REST API is broad and consistent, covering the entire PDF lifecycle – CRUD, page operations, merge/split, form fields, watermarking, and multi-format export.

Where the Risk Sits

- No automated test suite yet – no unit tests, no integration tests, nothing end-to-end.
- Deployment today means running a Windows batch script that starts Uvicorn locally; there's no Dockerfile and no CI/CD pipeline.
- auth.py falls back to a hardcoded JWT secret if the environment variable isn't set, and passwords are hashed with only 1,000 PBKDF2 iterations – both worth fixing before this goes anywhere near the public internet.
- Metadata lives in a single db.json file guarded by an in-process lock, which is fine for one server and won't survive scaling out to more than one.

2. Project Overview & Objectives

2.1 The Problem

Editing a PDF properly usually means opening a desktop app like Acrobat, or routing the file through some conversion pipeline. QT PDF Editor's whole premise is to skip that: upload once, and everything after – viewing, marking up, restructuring – happens against that same file, in the browser, with edits written straight back into a standards-compliant PDF rather than bounced through an intermediate format.

2.2 What It Sets Out To Do

- Render PDFs faithfully in the browser, fast enough that scrolling and zooming actually feel responsive.
- Cover the annotation types people actually reach for: text, highlight/underline/strikeout, freehand ink, shapes, images, sticky notes, hyperlinks.
- Support real in-place text editing rather than an overlay box pretending to be one, by detecting paragraph structure and reflowing edited text with the source document's own fonts.
- Handle document management, not just markup: rename, soft/hard delete, restore, rotate/insert/duplicate/reorder/delete/extract pages, split, merge.
- Gate access behind accounts, and offer export to PDF, PNG, or JPEG with control over page range and DPI.
- Keep a real undo/redo history at the document level, so a bad edit doesn't mean starting over.

2.3 Who It's For

Individuals and small teams who need to mark up, redline, fill in, or lightly restructure a PDF, and would rather not install anything to do it.

3. System Architecture

The system splits along the usual client/server line, and it sticks to that split cleanly: the client owns rendering and interaction, the server owns the actual PDF bytes and every structural change made to them. Nothing that affects document correctness is duplicated on the client – if you want to know what a document actually looks like, the server's copy is the answer.

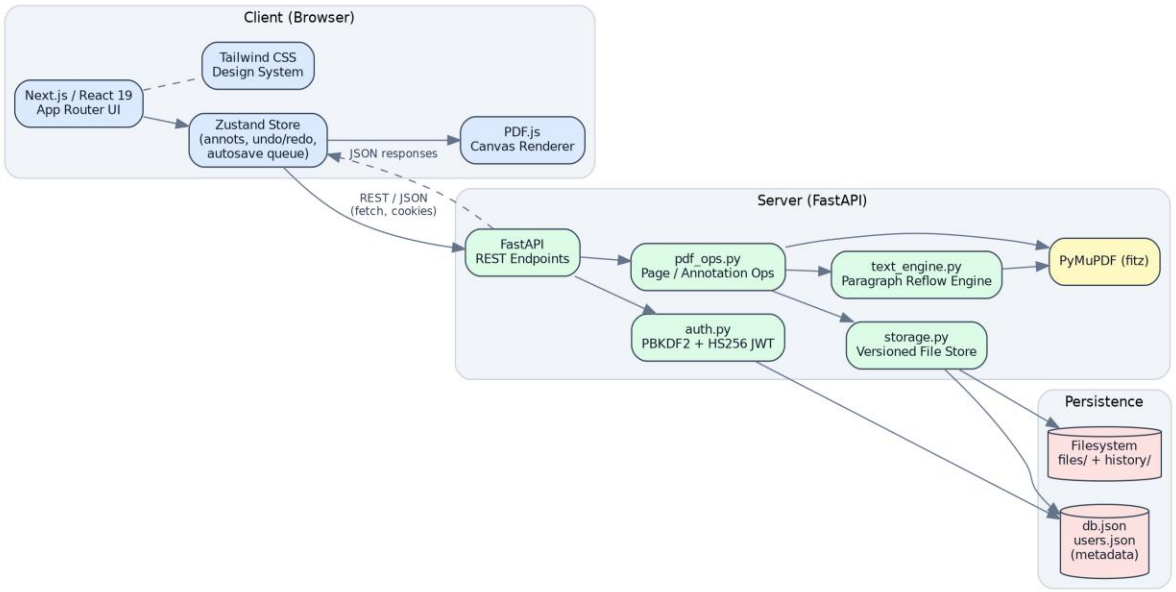

Figure 1 – QT PDF Editor system architecture

3.1 Frontend

Technology	Role
Next.js (React 19, App Router)	Routing for auth, editor, preview, and profile pages; the editor itself lives in client components
Zustand	Holds the editor's live state: the loaded document, annotation list, undo/redo stacks, tool options, panel layout, autosave queue
Tailwind CSS v4	Styling across the whole app, from the landing page to the editor chrome
PDF.js (pdfjs-dist)	Renders pages to <canvas> on the client – the only place the PDF actually gets parsed for display
Lucide React	The icon set used throughout the toolbar and sidebars

The editor itself (components/editor-studio/) is built around one Editor.tsx shell that pulls together a Toolbar, a LeftSidebar for pages and the outline, CanvasStage/PageView for the actual rendering, a RightSidebar for properties and the annotation list, an AnnotLayer overlay that handles interactive editing, and a StatusBar. A separate Previewer/PreviewTopBar pair handles the read-only view used for shared documents.

3.2 Backend

Technology	Role
FastAPI	The REST layer – routing, request validation via Pydantic, structured error handling

Technology	Role
Uvicorn	The ASGI server the FastAPI app runs on (directly, or via `python main.py --port`)
PyMuPDF (fitz)	Does the real work: all parsing, rendering-to-image, and mutation – annotations, pages, forms, redaction
fonttools	Used by the reflow engine to inspect embedded fonts, so it can decide whether to reuse one or fall back
python-multipart	Handles the multipart upload on `/api/files/upload`

Four modules do essentially all of the backend's work: main.py holds every HTTP route, auth.py owns the user store and JWT sign/verify logic, storage.py is the file-backed metadata store with per-file locking and undo/redo snapshotting, pdf_ops.py handles page- and annotation-level PDF operations, and text_engine.py is the paragraph detection and reflow engine behind in-place text edits.

3.3 Data & Storage Layer

There's no database here in the traditional sense. File metadata lives in backend/data/db.json, keyed by a 12-character hex id, with the PDF bytes sitting next to it in backend/data/files/{id}.pdf. Every mutating operation copies the pre-change PDF into backend/data/history/ before touching it, and storage.py keeps a capped, per-file undo/redo stack (20 entries) pointing at those snapshots. User accounts follow the same pattern in a flat users.json.

- A single module-level threading.Lock() serializes reads and writes of db.json, since FastAPI's sync endpoints run on a thread pool and can genuinely overlap.
- A second, per-file lock (storage.op_lock) additionally serializes concurrent edits against the same document, so two requests can't race on one PDF.
- That's a sound design for a single process, but it doesn't extend to multiple server instances – more on that in Sections 7 and 10.

4. Core Workflow: The Annotation Bake Pipeline

If there's one piece of this system worth understanding properly, it's how an edit made in the browser turns into a permanent, structurally correct change in the actual PDF. The team's own term for this is "baking" an edit, and it's a genuinely careful piece of engineering.

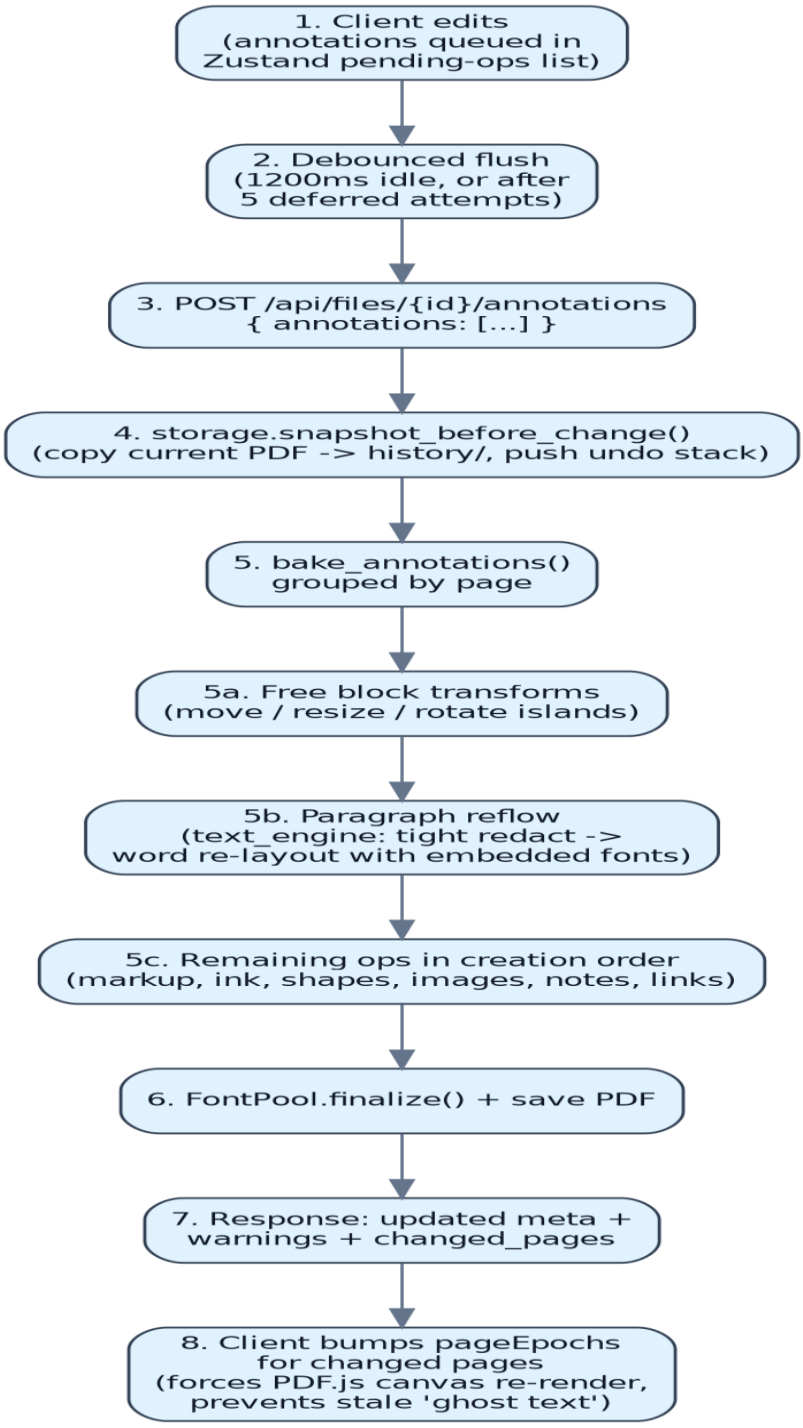

Figure 2 – Client edit to persisted PDF: the annotation bake sequence

4.1 Why Ordering Matters Here

bake_annotations() in pdf_ops.py doesn't just apply a batch of pending edits in the order they arrived – it processes them in three deliberate passes, because redaction is destructive to whatever's sitting in that page region at the time:

- Pass 1, free block transforms: text blocks the user has picked up and moved, resized, or rotated as floating islands. These redact their old spot and redraw somewhere else, so they need to happen before anything else touches the page.
- Pass 2, paragraph reflow: in-place text rewrites go next, because their redaction step would otherwise wipe out any annotation the user had visually placed over that same paragraph.
- Pass 3, everything else: highlights, ink, shapes, images, notes, links – applied in whatever order they were originally created.

That ordering is exactly what prevents the "ghost text" problem the team ran into during development. If a text edit's redaction ran after something had already been drawn on top of it – or if the client didn't force the canvas to actually re-render after the server rewrote the page – old glyph data could linger on screen until the next full repaint. The fix has two halves: the server-side pass ordering described above, and a client-side counterpart where the frontend bumps a per-page "epoch" counter (pageEpochs, in the Zustand store) for every page the server reports as changed. That forces PDF.js to throw away the old canvas bitmap for those pages instead of quietly compositing over it.

5. Notable Engineering Challenges & Resolutions

5.1 The Ghost Text Race Condition

Covered in detail in Section 4.1. The short version: a strict three-pass ordering on the server means redactions never wipe out content drawn after them, and a client-side render-epoch bump makes sure PDF.js discards any stale canvas for a page instead of layering new content over old.

5.2 Redaction Mask Rendering

The naive approach – redact a paragraph's whole bounding box before rewriting it – tends to bleed into the lines above and below, or leave faint edge artifacts. text_engine.py avoids this by computing a tight redaction rectangle for each individual visual line (_line_rect), shaving a small inset off the top and bottom of each line (_REDACT_INSET_V, 0.75pt, capped at 18% of line height) so redaction never touches a neighboring line. It then calls PyMuPDF's apply_redactions() with flags that preserve images and line art, so nothing outside the text itself gets disturbed. One wrinkle: redaction rebuilds the page's content stream and drops its font resources in the process, so the FontPool object deliberately forgets any font aliases it had registered for that page right after redaction runs (page_redacted()) – otherwise text reinserted afterward would point at font resources that no longer exist.

5.3 Font Family Detection & Reuse

When someone edits an existing paragraph, the goal is to keep using the document's own embedded font rather than quietly swapping in a generic PDF base-14 font, so the edit doesn't visually stick out. A FontPool, built fresh for each bake operation, checks – via fonttools – whether the embedded font's glyph set actually covers the characters in the new text before reusing it. If it doesn't, it falls back to the closest base-14 family (helv/tiro/cour, in regular, bold, italic, or bold-italic) instead.

5.4 Concurrency on a Single File

FastAPI runs its synchronous endpoints on a thread pool, which means two requests against the same document can genuinely execute at the same time. Every mutating endpoint takes out storage.op_lock(file_id) before touching the file, and snapshots the pre-change state first, so undo history stays coherent even when edits are coming in quickly – as they do from the debounced autosave queue on the client.

6. API Reference

Every route below lives in backend/main.py. Auth works through an httpOnly qt_token cookie – a signed HS256 JWT that expires after 24 hours – rather than a bearer token you'd attach manually. It gets set on register and login, and cleared on logout. CORS is currently locked down to localhost origins (http://localhost:3000, http://127.0.0.1:3000, and app://localhost), which is exactly right for local dev and exactly wrong for anything beyond it.

6.1 Authentication

Method	Path	Description
POST	/api/auth/register	Create a user (name, email, password of at least 6 characters); sets the qt_token cookie
POST	/api/auth/login	Check credentials and set the qt_token cookie
POST	/api/auth/logout	Clear the qt_token cookie
GET	/api/auth/me	Return the current session's user, or loggedIn: false

6.2 File Lifecycle

Method	Path	Description
POST	/api/files/upload	Multipart upload; confirms the PDF actually opens and records its page count
GET	/api/files	List files (pass include_deleted to also see soft-deleted ones)
GET	/api/files/{id}	Get metadata for one file
POST	/api/files/{id}/open	Mark a file as opened, updating opened_at for recency sorting
PATCH	/api/files/{id}	Rename a file
GET/HEAD	/api/files/{id}/content	Raw PDF bytes with a no-store cache header – what the PDF.js viewer actually fetches
GET	/api/files/{id}/download	Raw PDF bytes with a Content-Disposition attachment header

Method	Path	Description
DELETE	/api/files/{id}	Soft delete by default; add ?permanent=true to hard-delete the file and its history
POST	/api/files/{id}/restore	Bring a soft-deleted file back

6.3 Looking Inside a Document

Method	Path	Description
GET	/api/files/{id}/outline	The PDF's bookmark/outline tree
GET	/api/files/{id}/paragraphs?page=	Editable paragraphs the text engine detected on a page
GET	/api/files/{id}/notes	List sticky-note annotations already baked into the file
PATCH	/api/files/{id}/notes/{xref}	Edit a note's text
DELETE	/api/files/{id}/notes/{xref}	Delete a note
GET	/api/files/{id}/fields	List AcroForm fields – text, checkbox, choice
POST	/api/files/{id}/fields	Bulk-set form field values by xref

6.4 Editing & History

Method	Path	Description
POST	/api/files/{id}/annotations	Bake a batch of pending edits into the PDF – see Section 4
POST	/api/files/{id}/undo	Roll back to the most recent snapshot
POST	/api/files/{id}/redo	Re-apply whatever was just undone
POST	/api/files/{id}/watermark	Stamp a text watermark – color, opacity, rotation, page range all configurable

6.5 Page Operations

Method	Path	Description
POST	/api/files/{id}/pages/rotate	Rotate selected pages by a multiple of 90°
POST	/api/files/{id}/pages/delete	Delete selected pages
POST	/api/files/{id}/pages/insert	Insert a blank page after a given index
POST	/api/files/{id}/pages/duplicate	Duplicate selected pages

Method	Path	Description
POST	/api/files/{id}/pages/reorder	Reorder pages into a new sequence
POST	/api/files/{id}/pages/extract	Pull selected pages out into a new document
POST	/api/files/{id}/split	Split into multiple documents by page-range groups
POST	/api/files/merge	Merge two or more documents into one

6.6 Export

Method	Path	Description
GET	/api/files/{id}/export?format=&pages=&dpi=	Export as PDF, PNG, or JPEG – a single image, or a ZIP if you asked for more than one page

6.7 How Errors Come Back

- Every HTTP and validation error gets normalized to the same { error, detail } JSON shape by a couple of global exception handlers.
- 404 means the file id doesn't exist; 410 Gone means the metadata is there but the actual PDF bytes on disk have gone missing somehow.
- 409 shows up for state-dependent failures – nothing left to undo/redo, or a text edit whose geometry has gone stale.
- 422 means the request was valid JSON but PyMuPDF rejected the operation itself. bake_annotations wraps each item in its own try/except, so one bad annotation in a batch turns into a warning rather than failing everything else in the same request.

7. Security

What follows is just an honest look at where things stand today, based on the code as it exists on sprint1. None of this is unusual for an early-stage project – it's the kind of list you'd expect to see before a first production deploy, not a sign anything went wrong.

7.1 Where Things Stand

Area	Observation
JWT signing secret	auth.py falls back to a hardcoded string if the JWT_SECRET environment variable isn't set. Any deployment that forgets to set it is signing tokens with a secret that's sitting right there in the source.
Password hashing	PBKDF2-HMAC-SHA512 at 1,000 iterations. It's implemented correctly, but 1,000 iterations is low by current standards (OWASP's current guidance for PBKDF2-SHA512 is well north of 200,000) – cheap enough that a leaked user store would be crackable offline without much effort.
Session cookie	The JWT lives in an httpOnly, SameSite=Lax cookie, which is a reasonable default against basic XSS token theft. There's no explicit Secure flag yet, and no CSRF token on state-changing requests.
CORS	Locked to localhost origins right now, which is correct for development and needs revisiting the moment this goes anywhere else.
Who can access what	File routes aren't scoped to the logged-in user – anyone who knows a valid file id can hit it. There's no ownership check in storage.py or main.py yet.
Input handling	This part's in decent shape already: filenames go through a regex allow-list (_safe_name) before touching disk, and page ranges/rotation degrees get validated before use.
Secrets in source	No .env file in the repo (correctly gitignored), but the JWT fallback value ships in source, which is really the same problem wearing a different hat.
Transport	Everything currently assumes plain HTTP on localhost – TLS isn't part of the picture yet.

7.2 What I'd Fix First

- Make JWT_SECRET required at startup – fail loudly if it's missing, instead of quietly signing with a fallback.
- Bump PBKDF2 iterations way up, or switch to Argon2id, which is built specifically for password hashing.
- Add per-user ownership on file records and check it on every /api/files/* route, not just in the UI.
- Add CSRF protection for the cookie-based auth – a double-submit token, or SameSite=Strict where the workflow allows it.
- Put TLS in front of this before it's reachable from anywhere but localhost, and turn on the Secure cookie flag.
- Rate-limit the auth endpoints – right now nothing stops repeated login attempts.

8. Testing

8.1 Where Things Stand

There isn't a test suite yet – no test files, no runner config, no CI workflow. Right now, verifying something works means running the app and trying it by hand.

8.2 What I'd Actually Test

Backend (pytest)

- The pure functions in pdf_ops.py first – parse_ranges/parse_range_groups, hex2rgb, the fontname mapping. Fast, no PDF needed.
- The geometry helpers in text_engine.py (the _line_rect insets, the thresholds for splitting paragraphs) against a few small hand-built PDFs.
- End-to-end tests against bake_annotations using fixture PDFs – specifically, proving the pass ordering actually prevents ghost text, that undo/redo round-trips get you back to byte-identical pages, and that one bad annotation in a batch produces a warning instead of a 500.
- Contract tests (FastAPI's TestClient) for every route in Section 6, including the 404/409/410/422 cases specifically.

Frontend (Vitest / React Testing Library + Playwright)

- Store logic first – the undo/redo transitions, the autosave debounce (FLUSH_DELAY, MAX_SELECTED_DEFERS), and pageEpochs actually bumping after a bake response comes back.
- Component-level tests for AnnotLayer and PageView – switching tools, dragging and resizing a selected annotation.
- A handful of real end-to-end flows in Playwright: upload, annotate, save, reload, and confirm it stuck; edit text, undo, confirm the original comes back; merge two files then split them and check the result.

8.3 Wiring This Into CI

A GitHub Actions workflow on every PR would go a long way – pytest for the backend, lint plus the Vitest suite for the frontend, and a small Playwright smoke test against a preview build. None of it exists yet, so this is really a from-scratch addition rather than an extension of something already there.

9. Deployment

9.1 Where Things Stand

Right now the only deployment artifact in the repo is start-backends.bat – a Windows batch file that fires up the FastAPI backend with `uvicorn main:app --port 8001 --reload`, pointed at a hardcoded local Python path, and tells you to start the frontend separately with `npm run dev`. There's no Dockerfile, no compose file, no pipeline of any kind.

9.2 Getting to Something Deployable

Containers

- Backend: a slim Python image running Uvicorn (behind Gunicorn, or with multiple Uvicorn workers), with the files/history/db directories mounted on a persistent volume.
- Frontend: a proper `next build`, served either through the Next.js production server or as a static export depending on how much stays dynamic.
- A docker-compose.yml tying frontend, backend, and a reverse proxy (nginx or Caddy) together, with the proxy handling TLS.

The Data Layer

- Swap db.json/users.json for SQLite as the smallest possible change, or PostgreSQL if you're planning to run more than one instance – storage.py's interface (get/create/update/list_files/soft_delete/hard_delete) is narrow enough that this shouldn't be a big rewrite.
- Move PDF files and history snapshots to object storage once there's more than one backend instance in the picture, since right now everything assumes one shared local filesystem.

Config & Environment

- Pull JWT_SECRET, allowed CORS origins, and storage paths out into environment variables, validated at startup rather than discovered at runtime.
- Add structured logging and wire a readiness probe off the existing /api/health and /health endpoints – they're already there, just not being used for anything yet.

Scaling

- The threading.Lock()-based per-file locking only works within a single process. Running more than one instance needs a real distributed lock – Postgres row locks or Redis both work – or sticky routing so the same file always lands on the same instance.

10. What's Next

10.1 Concurrency & Locking

The per-process file lock needs to become a distributed one – a Redis- or Postgres-backed lock, or a queue per file – before this can run on more than one backend instance. It's also the prerequisite for anything like real-time multi-user editing on the same document down the line.

10.2 Font Subsetting

Right now the font pool reuses whole embedded fonts when they cover what's needed. Subsetting with fonttools would mean only embedding the glyphs actually used after an edit – this matters most for documents with big CJK or symbol fonts, where editing a handful of characters shouldn't force re-embedding the entire font.

10.3 Canvas Virtualization

Pages currently render through PDF.js onto canvas elements one by one. For a long document, only mounting and rendering pages near the viewport – the store already half-supports this via viewMode/currentPage – would cut memory use and keep scrolling smooth.

10.4 Other Things Worth Doing

- Real-time collaborative editing (operational transform or CRDTs), building on the concurrency work above.
- OCR for scanned or image-only PDFs, so paragraph detection and text editing work even without a real text layer.
- Actual digital signatures – visible and cryptographic – beyond the free-form "sign" tool that's already reserved in the frontend's Tool type.
- An accessibility pass on the editor – keyboard navigation for placing annotations, ARIA labels on the canvas overlay.
- Proper API versioning (/api/v1/...) before anyone outside the project starts depending on these routes.

11. Appendix

11.1 Repository Snapshot

Item	Value
Repository	quantiradev/qt_pdf-editor
Branch reviewed	sprint1
Frontend framework	Next.js 16.2.10, React 19.2.4
Backend framework	FastAPI (>=0.111), PyMuPDF (>=1.24.9)
Notable commits	Initial commit, favicon/meta title, Sprint 1 auth, previewer added, editor tool fixes, editor upgrade

11.2 Glossary

- Bake – permanently applying a pending client-side edit into the actual stored PDF.
- Ghost text – the rendering glitch where old text stays visible after an edit because of a redaction/render-ordering race.
- Reflow – re-wrapping edited paragraph text inside its box using the source document's own font metrics.
- Snapshot – a full copy of a PDF's bytes taken right before a mutating operation, which is what makes undo/redo possible.